R_control_flow

Supercomputer Center, Westlake University.

2019/10/10

Control Structures

Control structures in R allow you to control the flow of execution of the program, depending on runtime conditions. Common structures are

- if, else: testing a condition
- for: execute a loop *for* a fixed number of times
- while: execute a loop *while* a condition is true
- repeat: execute an infinite loop
- break: *break* the execution of a loop
- next: skip an interation of a loop
- return: exit a function and give a result

Most control structures were designed for non-interactive mode and longer expressions.

if .. (else) ..

The else clause is not necessary.

```
> x <- rbinom(1,10,0.5)
> y <- if (x > 3) {
+   10
+ } else {
+   0
+ }
> x
```

```
## [1] 8
```

```
> y
```

```
## [1] 10
```

for

For loops are most commonly used for iterating over the elements of an object (list, vector, etc.)

```
> x <- letters[1:4]
> for (i in 1:4) {
+   print(x[i])
+ }
```

```
## [1] "a"
## [1] "b"
## [1] "c"
## [1] "d"
```

for (cont'd)

```
> for (i in seq_along(x)) {
+   print(x[i])
+ }

## [1] "a"
## [1] "b"
## [1] "c"
## [1] "d"
```

for (cont'd)

```
> for (i in x) {
+   print(i)
+ }

## [1] "a"
## [1] "b"
## [1] "c"
## [1] "d"
```

for (cont'd)

The for loops can be nested. Nesting beyond 2-3 levels is often very hard to read.

```
> x <- matrix(1:6,2,3)
> for (i in seq_len(nrow(x))) {
+   for (j in seq_len(ncol(x))) {
+     print(x[i,j])
+   }
+ }
```

```
## [1] 1
## [1] 3
## [1] 5
## [1] 2
## [1] 4
## [1] 6
```

while

While loops begin by testing a condition. It can potentially result in infinite loops if not written properly. Conditions are always evaluated from left to right. **Use with care!**

```
> z <- 5
> while (z >= 3 & z <= 10) {
+   print(z)
+   coin <- rbinom(1,1,0.5)
+   # random walk
+   if (coin == 1) {
+     z <- z + 1
+   } else {
+     z <- z - 1
+   }
+ }
```

repeat

Repeat initiates an **infinite** loop; these are not commonly used in statistical applications but they do have their uses. The only way to exit a repeat loop is to call *break*.

```
> x <- 1
> tol <- 1e-5
> repeat {
+   y <- rnorm(1)
+   if (abs(y - x) < tol) {
+     break
+   } else {
+     x <- y
+   }
+ }
> c(x,y)
```

```
## [1] -0.3280948 -0.3281030
```

next, return

next is used to skip an iteration of a loop. *return* signals that a function should exit and return a given value.

```
> x <- 1:10
> ex_fun <- function(x) {
+   return(x - 1)
+ }
> for (i in seq_along(x)) {
+   if (i <= 5) {
+     next
+   }
+   x[i] <- ex_fun(x[i])
+ }
> x
```

```
##  [1] 1 2 3 4 5 5 6 7 8 9
```

Summary

- Control structures like *if*, *while*, and *for* allow you to control the flow of an R program
- Infinite loops should generally be avoided, even if they are theoretically correct

Functions

Functions

Functions are created using the function() directive and are stored as R objects just like anything else. In particular, they are R objects of class "function".

```
f <- function(<arguments>) {
  ## Do something interesting
}
```

Functions in R are "first class objects", which means that they can be treated much like any other R object. Importantly,

- Functions can be passed as arguments to other functions
- Functions can be nested, so that you can define a function inside of another function

The return value of a function is the last expression in the function body to be evaluated.

Function Arguments

Functions have *named arguments* which potentially have *default values*.

- The *formal arguments* are the arguments included in the function definition
- The formula function returns a list of all the formal arguments of a function
- Not every function call in R makes use of all the formal arguments
- Function arguments can be *missing* or might have default values

Argument Matching

R functions arguments can be matched positionally or by name. So the following calls to sd are all equivalent

```
> mydata <- rnorm(100)
> sd(mydata)
```

```
## [1] 0.9335628
```

```
> sd(x=mydata)
```

```
## [1] 0.9335628
```

```
> sd(na.rm=F,x=mydata)
```

```
## [1] 0.9335628
```

```
> sd(na.rm=F,mydata)
```

```
## [1] 0.9335628
```

Even though it's legal, I don't recommend messing around with the order of the arguments too much,since it can lead to some confusion.

Argument Matching (cont'd)

You can mix positional matching with matching by name. When an argument is matched by name, it is "taken out" of the argument list and the remaining unnamed arguments are matched in the order that they are listed in the function definition.

```
> args(lm)

function (formula, data, subset, weights, na.action, method = "q
    model = TRUE, x = FALSE, y = FALSE, qr = TRUE, singular.ok =
    contrasts = NULL, offset, ...)
NULL
```

The follwing two calls are equivalent.

```
lm(data = mydata, y ~ x, model = FALSE, 1:100)
lm(y ~ x, mydata, 1:100, model = FALSE)
```

Argument Matching (cont'd)

- Most of the time, named arguments are useful on the command line when you have a long argument list and you want to use the defualts for everything except for an argument near the end of the list
- Named arguments also help if you can remember the name of the argument and not its position on the argument list (plotting is a good example)

Argument Matching (cont'd)

Function arguments can also be *partially* matched, which is useful for interactive work. The order of operations when given an argument is

- Check for exact match for a named argument
- Check for a partial match
- Check for a positional match

Defining a Function

```
f <- function(a, b = 1, c = 2, d = NULL) {

}
```

In addition to not specifying a default value, you can also set an argument value to NULL.

Lazy Evaluation

Arguments to functions are evaluated *lazily*, so they are evaluted only as needed.

```
f <- function(a, b) {
  a ^ 2
}
f(2)
```

```
## [1] 4
```

This function never actually uses the argument b, so calling f(2) will not produce an error because 2 get positionally matched to a.

Lazy Evaluation (cont'd)

Another example

```
> f <- function(a, b) {
+   print(a)
+   print(b)
+ }
```

```
> f(45)
+ [1] 45
+ Error in print(b) : argument "b" is missing, with no default
```

Notice that "45" got printed first before the error was triggered. This is because b did not have to be evaluated until after print(a). Once the function tried to evaluate print(b) it had to throw an error.

The "..." Argument

The ... argument indicate a variable number of arguments that are usually passed on to other functions.

- ... is often used when extending another function and you don't want to copy the entire argument list of the original function

```
myplot <- function(x, y, type = "l", ...) {
  plot(x, y, type = type, ...)
}
```

- Generic functions use ... so that extra arguments can be passed to methods (more on this later).

```
> mean
```

```
## function (x, ...)
## UseMethod("mean")
## <bytecode: 0x55a8e5830458>
## <environment: namespace:base>
```

The "..." Argument

The ... argument is also necessary when the number of arguments passed to the function cannot be known in advance.

```
> args(paste)

## function (..., sep = " ", collapse = NULL)
## NULL

> args(cat)

## function (..., file = "", sep = " ", fill = FALSE, labels = N
##     append = FALSE)
## NULL
```

Arguments Coming After the "..."

One catch with ... is that any arguments that appear *after* ... on the argument list must be named explicitly and cannot be partially matched.

```
> args(paste)

## function (..., sep = " ", collapse = NULL)
## NULL

> paste("a", "b", sep = ":")

## [1] "a:b"

> paste("a", "b", se = ":")

## [1] "a b :"
```

Scoping Rules

A Diversion on Binding Values to Symbol

How does R know which value to assign to which symbol? When I type

```
> lm <- function(x) {
+   x * x
+ }
> lm
```

```
## function(x) {
##   x * x
## }
```

how does R know what value to assign to the symbol lm? Why doesn't it give it the value of lm that is in the **stats** package?

A Diversion on Binding Values to Symbol (cont'd)

When R tries to bind a value to a symbol, it searches through a series of environments to find the appropriate value. When you are working on the command line and need to retrieve the value of an R object, the order is roughly

1. Search the global environment for a symbol name matching the one requested.
2. Search the namespaces of each of the packages on the search list

```
> search()
```

```
## [1] ".GlobalEnv"        "package:stats"
## [3] "package:graphics"  "package:grDevices"
## [5] "package:utils"     "package:datasets"
## [7] "package:methods"   "Autoloads"
## [9] "package:base"
```

Binding Values to Symbol

- The *global environment* or the user's workspace is always the first element of the search list and the **base** package is always the last.
- The order of the packages on the search list matters!
- User's can configure which packages get loaded on startup so you cannot assume that there will be a set list of packages available.
- When a user loads a package with library the namespace of that package gets put in position 2 of the search list (by default) and everything else gets shifted down the list.
- Note that R has separate namespaces for functions and non-functions so it's possible to have an object named c and a function named c.

Scoping Rules

The scoping rules for R are the main feature that make it different from the original S language.

- The scoping rules determine how a value is associated with a free variable in a function
- R uses *lexical scoping* or *static scoping*. A common alternative is *dynamic scoping*.
- Related to the scoping rules is how R uses the *search list* to bind a value to a symbol
- Lexical scoping turns out to be particularly useful for simplifying statistical computations

Lexical Scoping

Consider the following function.

```
f <- function(x, y) {
  x ^ 2 + y / z
}
```

This function has 2 formal arguments x and y. In the body of the function there is another symbol z. In this case z is called a free variable.

The scoping rules of a language determine how values are assigned to free variables.

Free variables are not formal arguments and are not local variables (assigned insided the function body)

Lexical Scoping (cont'd)

Lexical scoping in R means that

> *the values of free variables are searched for in the environment in which the function was defined.*

What is an environment?

- An *environment* is a collection of (symbol, value) paires, i.e. x is a symbol and 3.14 might be its value
- Every environment has a parent environment; it is possible for an environment to have multiple "children"
- The only environment without a parent is the empty environment
- A function + an environment = a *closure* or *function closure*

Lexical Scoping (cont'd)

Searching for the value for a free variable:

- If the value of a symbol is not found in the environment in which a function was defined, then the search is continued in the *parent environment*.
- The search continues down the sequence of parent environments until we hit the *top-level environment*; this usually the global environment (workspace) or the namespace of a package.
- After the top-level environment, the search continues down the search list until we hit the *empty environment*.
- If a value for a given symbol cannot be found once the empty environment is arrived at, then an error is thrown.

Lexical Scoping (cont'd)

Why does all this matter?

- Typically, a function is defined in the global environment, so that the values of free variables are just found in the user's workspace
- This behavior is logical for most people and is usually the "right thing" to do
- However, in R you can have functions defined *inside other functions*
  - Languages like C don't let you do this
- Now thins get interesting – In this case the environment in which a function is defined is the body of another function!

Lexical Scoping (cont'd)

```
make.power <- function(n) {
  pow <- function(x) {
    x ^ n
  }
  pow
}
```

This function returns another function as its values.

```
> cube <- make.power(3)
> square <- make.power(2)
> cube(3)
```

```
## [1] 27
```

```
> square(3)
```

```
## [1] 9
```

Exploring a Function Closure

What's in a function's environment?

```
> ls(environment(cube))
## [1] "n"   "pow"
> get("n", environment(cube))
## [1] 3
> ls(environment(square))
## [1] "n"   "pow"
> get("n", environment(square))
## [1] 2
```

Lexical vs. Dynamic Scoping

```
y <- 10
f <- function(x) {
  y <- 2
  y ^ 2 + g(x)
}
g <- function(x) {
  x * y
}
```

What is the value of

f(3)

Lexical vs. Dynamic Scoping (cont'd)

- With lexical scoping the value of y in the function g is looked up in the environment in which the function was defined, in this case the global environment, so the value of y is 10.
- With dynamic scoping, the value of y is looked up in the environment from which the function was *called* (sometimes referred to as the *calling environment*).
  - In R the calling environment is known as the *parent frame*

  So the value of y would be 2

Lexical vs. Dynamic Scoping (cont'd)

When a function is *defined* in the global environment and is subsequently *called* from the global environment, then the defining environment and the calling environment are the same. This can sometimes give the appearance of dynamic scoping.

```
> g <- function(x) {
+   a <- 3
+   x + a + y
+ }
```

```
> g(2)
```

Error in g(2): object "y" not found

```
> y <- 3
> g(2)
```

```
## [1] 8
```

Other Languages

Other languages that support lexical scoping

- Scheme
- Perl
- Python
- Common Lisp (all languages converge to Lisp)

Consequences of Lexical Scoping

- In R, all objects must be stored in memory
- All functions must carry a pointer to their respective defining environmnents, which could be anywhere
- In S-PLUS, free variables are always looked up in the global workspace, so everything can be stored on the disk because the "defining environment" of all functions is the same.

Loop Function

Looping on the Command Line

Writing for, while loops is useful when programming but not particularly easy when working interactively on the command line. There are some functions which implement looping to make life easier.

- lapply: Loop over a list and evaluate a function on each element
- sapply: Same as lapply but try to simplify the result
- apply: Apply a function over the margins of an array
- tapply: Apply a function over subsets of a vector
- mapply: Multivariate version of lapply

An auxiliary function split is also useful, particularly in conjunction with lapply.

lapply

lapply takes three arguments: a list X, a function (or the name of a function) FUN, and other arguments via its ... argument. If X is not a list, it will be coerced to a list using as.list.

```
> lapply
```

```
## function (X, FUN, ...)
## {
##     FUN <- match.fun(FUN)
##     if (!is.vector(X) || is.object(X))
##         X <- as.list(X)
##     .Internal(lapply(X, FUN))
## }
## <bytecode: 0x55a8e0cdb070>
## <environment: namespace:base>
```

The actual looping is done internally in C code.

lapply (cont'd)

lapply always returns a list, regardless of the class of the input.

```
> x <- list(a = 1:5, b = rnorm(10))
> lapply(x, mean)

## $a
## [1] 3
##
## $b
## [1] 0.0191478
```

lapply (cont'd)

```
> x <- list(a = 1:4, b = rnorm(10), c = rnorm(20, 1),
+           d = rnorm(100, 5))
> lapply(x, mean)
```

```
## $a
## [1] 2.5
##
## $b
## [1] 0.3095494
##
## $c
## [1] 0.7587484
##
## $d
## [1] 4.872382
```

lapply (cont'd)

```
> x <- 1:4
> lapply(x, runif)

## [[1]]
## [1] 0.8129194
##
## [[2]]
## [1] 0.5876352 0.3684265
##
## [[3]]
## [1] 0.9295803 0.1944565 0.8744049
##
## [[4]]
## [1] 0.18365348 0.08631302 0.85779297 0.86083740
```

lapply (cont'd)

```
> x <- 1:4
> lapply(x, runif, min = 0, max = 10)

## [[1]]
## [1] 9.995981
##
## [[2]]
## [1] 5.702959 6.722939
##
## [[3]]
## [1] 7.158226 4.746997 7.164781
##
## [[4]]
## [1] 0.4076856 3.8878925 2.1594875 8.2379733
```

lapply (cont'd)

lapply and friends make heavy use of *anonymous function*.

```
> x <- list(a = matrix(1:4, 2, 2), b = matrix(1:6, 3, 2))
> x
```

```
## $a
##      [,1] [,2]
## [1,]    1    3
## [2,]    2    4
##
## $b
##      [,1] [,2]
## [1,]    1    4
## [2,]    2    5
## [3,]    3    6
```

lapply (cont'd)

An anonymous function for extracting the first column of each matrix.

```
> lapply(x, function(elt) elt[,1])
```

```
## $a
## [1] 1 2
##
## $b
## [1] 1 2 3
```

sapply

sapply will try to simplify the result of lapply if possible.

- If the result is a list where every element is length 1, then a vector is returned
- If the result is a list where every element is a vector of the same length (> 1), a matrix is returned.
- If it can't figure things out, a list is returned.

sapply (cont'd)

```
> x <- list(a = 1:4, b = rnorm(10), c = rnorm(20, 1),
+           d = rnorm(100, 5))
> sapply(x, mean)
```

```
##         a         b         c         d
## 2.5000000 0.2172274 1.0342581 4.9504533
```

```
> mean(x)
```

```
## Warning in mean.default(x): argument is not numeric or
## logical: returning NA
```

```
## [1] NA
```

Q&N

Questions & Thinking

1. Write a function that calculates the correlation between **Sepal.Length** and **Petal.Length** for iris dataset
2. Calculates median of each feature by **Species** using *for*, *sapply*, and other method for iris dataset.

```
head(iris)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1          5.1         3.5          1.4         0.2  setosa
## 2          4.9         3.0          1.4         0.2  setosa
## 3          4.7         3.2          1.3         0.2  setosa
## 4          4.6         3.1          1.5         0.2  setosa
## 5          5.0         3.6          1.4         0.2  setosa
## 6          5.4         3.9          1.7         0.4  setosa
```

Next...

- Other **apply* functions
- Debugging Tools
- Simulation
- Plotting: Base Graphics, Lattice Graphics and ggplot2